Étude & Plan de réalisation — Portail E-Payment

Nov 2, 2026 · @lalle

1. Synthèse du projet

Le portail E-Payment est une plateforme web où le **contrat signé est la seule source des droits** : un abonné ne voit et n'utilise que les wallets et cartes inscrits dans son contrat actif. La conception (v1.0, novembre 2026) est solide sur le fonctionnel et la sécurité ; il reste 6 décisions métier à trancher avant de figer l'architecture (section 5).

Objectifs : centraliser abonnés, contrats et moyens de paiement ; dématérialiser la contractualisation par signature électronique ; automatiser l'ouverture et la révocation des droits ; permettre les paiements ; tracer et reporter les transactions.

Périmètre de la démonstration (workflow à démontrer) : 3 moyens de paiement — **Bankily** et **Masrvi** (wallets) et **Visa** (carte). Sedad et Mastercard, cités dans la conception, sont à prévoir dans le catalogue mais hors démo.

Élément	Démo (MVP)	Cible complète
Moyens de paiement	Bankily, Masrvi, Visa	+ Sedad, Mastercard, autres partenaires
Espaces	Administrateur + Abonné	Idem, avec rôles admin fins
Signature	Parcours intégré simple, avec preuves	Prestataire externe certifié si exigé
Paiements	Sandbox fournisseurs	Production + rapprochement, remboursements
Reporting	Tableaux de bord de base	Reporting complet et audit

2. Acteurs et besoins fonctionnels

Deux profils utilisent le portail, et cinq systèmes externes interviennent. Tout contrôle d'accès se fait côté serveur, jamais par simple masquage d'écran.

Acteur	Rôle	Fonctions clés
Administrateur	Gestion centrale	Tableau de bord, abonnés, catalogue des moyens, modèles et contrats, droits, transactions, paramètres
Abonné	Utilisateur final	Consulter/signer le contrat, voir ses moyens autorisés, payer, historique et reçus, profil et sessions
Fournisseurs wallets	Externe	APIs Bankily, Masrvi (paiement, statut, webhook)
Passerelle bancaire	Externe	Paiement Visa tokenisé, 3-D Secure
Prestataire de signature	Externe ou interne	Signature, preuves, horodatage
SMS / email	Externe	Notifications (contrat disponible, activation, paiement)

Espace Administrateur — modules

1. Tableau de bord : abonnés (total, actifs, suspendus), contrats par statut, volume et montant des transactions, répartition par moyen.
2. Abonnés : créer, modifier, activer, suspendre, désactiver ; champs référence, type (physique/morale), nom ou raison sociale, NNI/NIF, téléphone, email, adresse, statut.
3. Moyens de paiement : catalogue wallets et cartes, paramètres d'intégration, limites, supervision de disponibilité.
4. Contrats : modèles, sélection des moyens inclus, dates, frais et plafonds, génération PDF, envoi, suivi de signature, archivage.

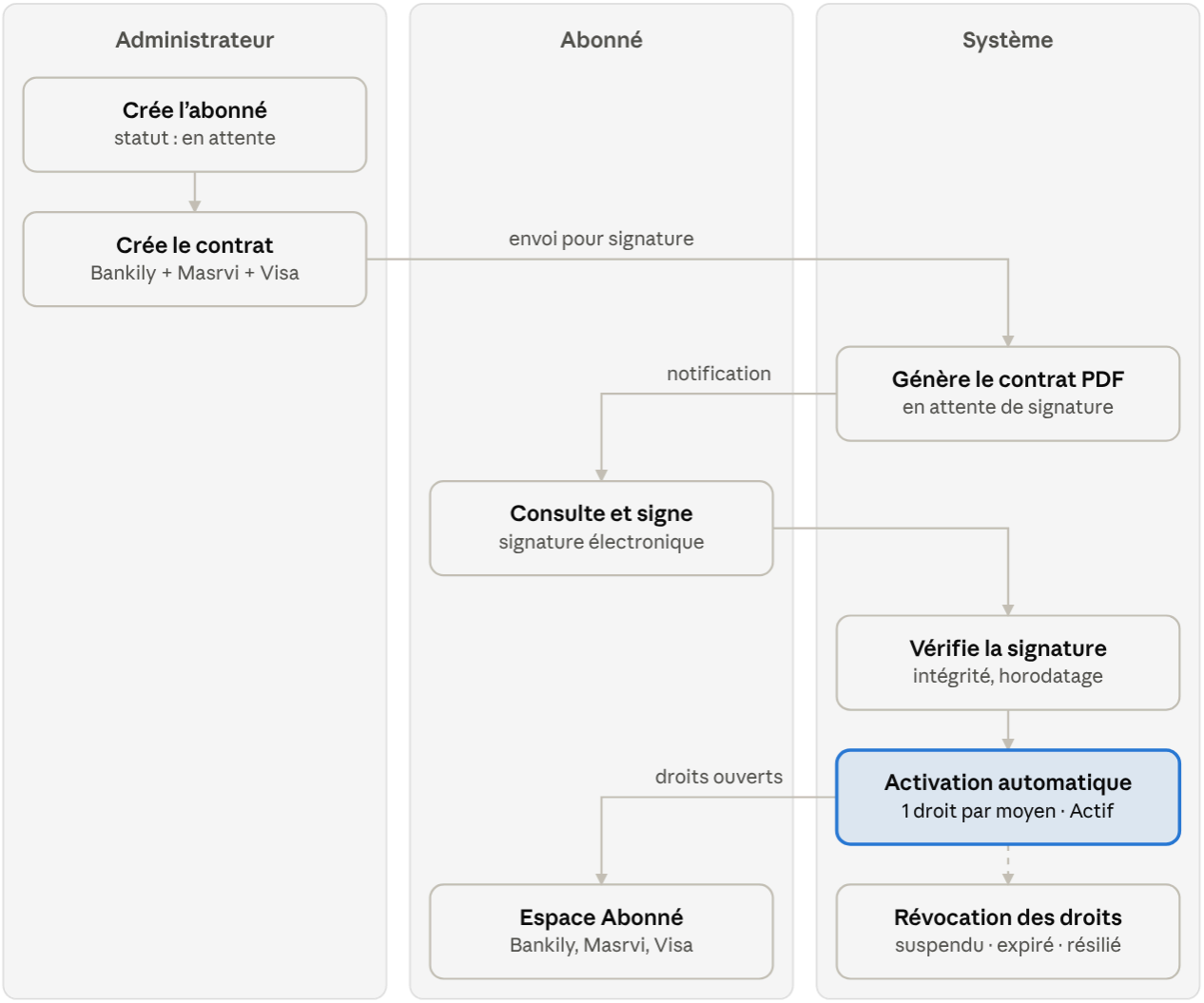
Espace Abonné — modules

1. Tableau de bord : statut du contrat, validité, moyens autorisés, dernières transactions, notifications.
2. Mon contrat : consultation, téléchargement, signature électronique.
3. Mes moyens de paiement : uniquement ceux du contrat actif, avec instructions.
4. Historique : référence, date, moyen, montant, statut, reçu.
5. Profil : coordonnées, mot de passe, sessions, préférences de notification.

3. Workflow principal à démontrer

Le scénario de démo enchaîne 7 étapes entre l'administrateur, l'abonné et le système ; le point critique est l'activation automatique, qui transforme le contrat signé en droits sur Bankily, Masrvi et Visa.

Le contrat signé, et lui seul, ouvre les droits de paiement



workflow principal · 3 acteurs, 8 étapes

Aucun paiement n'est possible avant l'étape « Activation automatique » ; toute suspension, expiration ou résiliation révoque ensuite les droits sans effacer l'historique.

Statut du contrat	Accès aux paiements
Brouillon	Non
En attente de signature	Non (consultation et signature seulement)
Signé / en attente d'activation	Non

Statut du contrat	Accès aux paiements
Actif	Oui, pendant la période de validité
Suspendu, expiré, résilié	Non (droits désactivés)

4. Architecture, données et sécurité

L'architecture proposée est un monolithe modulaire Spring Boot derrière une API REST, avec un adaptateur par fournisseur de paiement. C'est le bon choix pour démarrer : un seul déploiement, des modules bien séparés qu'on pourra extraire plus tard.

Couche	Choix	Recommandation
Frontend	Angular ou React	Angular retenu ; une seule app avec deux espaces séparés par rôle
Backend	Java / Spring Boot, REST	Spring Security + JWT, Spring Data JPA, Flyway pour les migrations
Base de données	PostgreSQL	Contraintes d'unicité pour l'idempotence des paiements et des droits
Documents	Stockage objet sécurisé	MinIO (S3) : contrats PDF, versions signées, preuves
Secrets	Coffre-fort	Vault ou équivalent pour les clés API fournisseurs
Externes	Wallets, passerelle, signature, SMS/email	Interface commune `PaymentProvider` + un adaptateur par fournisseur

Modules backend (10) : Auth & Authorization · Subscriber · Contract · E-Signature · Payment Methods · Entitlements · Payment Gateway · Transactions · Notification · Audit.

Modèle de données (13 entités) : User, Role/Permission, Subscriber, Contract_Template, Contract, Payment_Method, Contract_Payment_Method, Signature, Subscriber_Entitlement, Payment_Provider, Transaction, Notification, Audit_Log.

Relations clés :

- Subscriber 1—N Contract ; Contract N—N Payment_Method (via Contract_Payment_Method).
- Contract actif → N Subscriber_Entitlement (un par moyen de paiement, avec période et statut).

- Transaction → un abonné, un moyen, et le contrat qui a autorisé l'accès.

Cœur technique : le moteur de droits. À l'activation, il crée un droit par moyen inscrit au contrat ; à la suspension, l'expiration ou la résiliation, il les désactive. Il doit être idempotent (rejouer l'activation ne crée pas de doublon) et chaque tentative de paiement revérifie le droit côté serveur. Un job planifié quotidien gère les expirations.

Sécurité — points non négociables

- HTTPS partout ; mots de passe hachés (BCrypt/Argon2) ; MFA pour les administrateurs.
- Aucun numéro de carte complet ni CVV stocké : tokenisation par la passerelle (réduit le périmètre PCI DSS).
- Webhooks fournisseurs vérifiés par signature, avec protection anti-rejeu.
- Clé d'idempotence sur chaque demande de paiement pour éviter les doubles débits.
- Journal d'audit des actions sensibles, sans données de carte ni secrets dans les logs.
- Hash du PDF signé conservé pour prouver l'intégrité du contrat.

Choix techniques retenus

La stack retenue est **Angular + Spring Boot + PostgreSQL + Keycloak**. C'est le standard des portails bancaires et back-offices métier : typée de bout en bout, structurée, et pensée pour des applications riches en formulaires, tableaux et rôles.

Domaine	Choix retenu	Pourquoi ce choix
Frontend	**Angular 20** (TypeScript, composants standalone, signals)	Cadre complet (routing, formulaires réactifs, HTTP, guards) et structure imposée : idéal pour un back-office à 2 espaces et une équipe Java. React demanderait d'assembler ces briques soi-même.
Composants UI	**PrimeNG**	Tableaux avec tri, filtres et pagination, formulaires, graphiques prêts pour les tableaux de bord admin
Backend	**Java 21 + Spring Boot 3** (API REST)	Stabilité, écosystème sécurité et paiement, déjà prévu par la conception

Domaine	Choix retenu	Pourquoi ce choix
Authentification	**Keycloak** (OpenID Connect) + Spring Security	MFA admin, politique de mots de passe, gestion et révocation des sessions fournies sans développement
Base de données	**PostgreSQL 16** + Flyway	Transactions fiables, contraintes d'unicité pour l'idempotence, migrations versionnées
Accès aux données	Spring Data JPA (Hibernate)	Standard Spring, productif pour 13 entités
Génération PDF	**Thymeleaf + OpenHTMLtoPDF**	Modèles de contrat écrits en HTML, faciles à modifier
Signature électronique	**Intégrée au portail** : OTP par SMS + scellement PAdES du PDF (bibliothèque EU DSS)	Ne dépend d'aucun prestataire indisponible localement ; un adaptateur permet de brancher un prestataire externe plus tard
Stockage documents	**MinIO** (compatible S3)	Auto-hébergeable, chiffrement, politique de rétention
Secrets	**HashiCorp Vault**	Clés API Bankily, Masrvi, passerelle isolées des données métier
Paiement Visa	**Page de paiement hébergée** de la passerelle + tokenisation	Le portail ne voit jamais le numéro de carte : périmètre PCI DSS minimal (SAQ A)
Tâches planifiées	Spring Scheduler + ShedLock	Expiration des contrats, rapprochement, sans doublon en multi-instance
Notifications	Spring Mail (SMTP) + API SMS d'un opérateur local	Simple ; adaptateur SMS remplaçable
Tests	JUnit 5, Testcontainers, Playwright	Tests réels sur PostgreSQL et tests de bout en bout du workflow
Déploiement	**Docker + Docker Compose**, Nginx en reverse proxy	Suffisant pour la démo et la première production ; Kubernetes seulement si la charge l'exige

Domaine	Choix retenu	Pourquoi ce choix
CI/CD	GitLab CI	Auto-hébergeable, pipelines build, tests, images Docker
Supervision	Prometheus + Grafana, logs Loki	Disponibilité des fournisseurs et erreurs de paiement visibles
Architecture	**Monolithe modulaire** (Spring Modulith)	Un seul déploiement, modules isolés, découpage en services possible plus tard

Hypothèses métier retenues (à faire confirmer par l'organisation, mais le développement peut avancer dessus) :

Question	Choix retenu	Raison
Rôle du portail	L'abonné **paie**	Correspond au workflow à démontrer ; l'encaissement pourra être ajouté
Modèle des moyens	**Compte propre à chaque abonné** (son wallet, sa carte)	Responsabilités claires, pas de fonds centralisés à gérer
Plusieurs contrats	**Un seul contrat actif** par abonné ; un avenant le remplace	Supprime les règles de cumul et de priorité
Fuseau horaire	**Africa/Nouakchott** (UTC+0)	Dates de validité sans ambiguïté
Devise	**MRU** (ouguiya) ; devises carte selon la passerelle	Devise locale des wallets

5. Risques et questions ouvertes

Le risque principal est externe : l'accès aux APIs et sandbox de Bankily, Masrvi et de la passerelle Visa. Il faut lancer ces demandes dès la semaine 1, car leur délai ne dépend pas de l'équipe.

Risque	Impact	Parade
Accès tardif aux APIs Bankily / Masrvi / Visa	Bloque les phases 6–7	Demandes dès le début ; adaptateurs simulés (mock) pour avancer et pour la démo

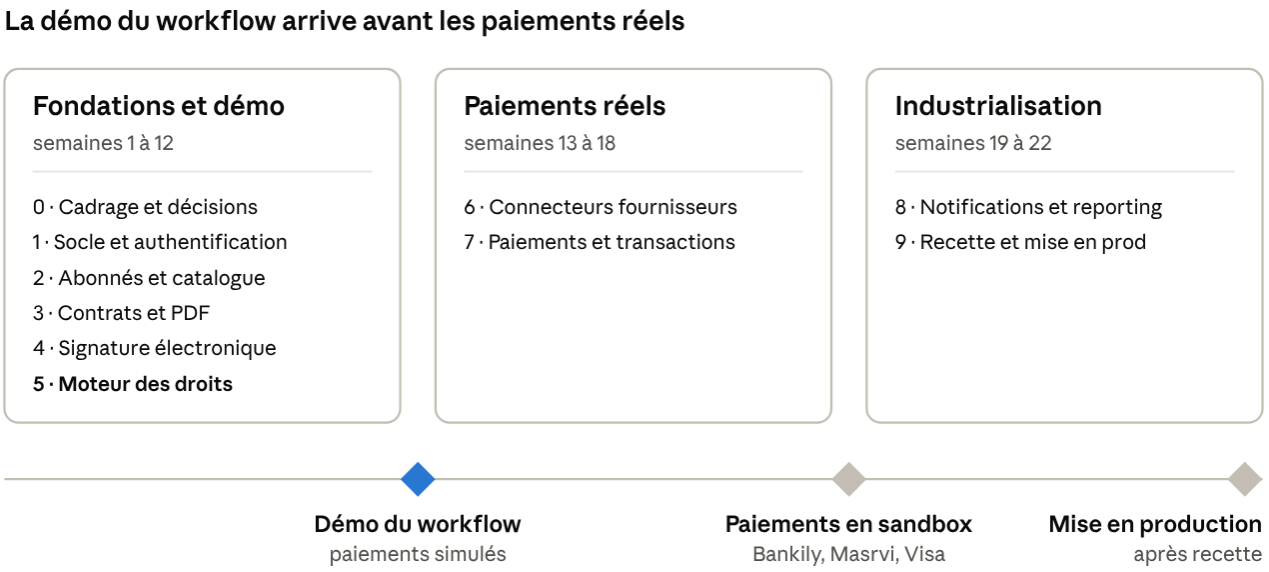
Risque	Impact	Parade
Signature non reconnue juridiquement	Contrats contestables	Valider le mode de signature avec le juridique ; conserver hash, horodatage, IP, OTP
Double débit ou statut incohérent	Perte financière, litiges	Idempotence, statut interne distinct du statut fournisseur, rapprochement
Droits ouverts trop tôt ou non révoqués	Accès non autorisé	Moteur de droits central + contrôle serveur à chaque paiement + tests dédiés
Périmètre PCI DSS	Coût de conformité	Paiement carte sur page hébergée de la passerelle, tokenisation
Plusieurs contrats par abonné	Ambiguïté des droits	Définir règles de cumul, priorité et révocation

Décisions à obtenir avant de figer l'architecture

- [x] ~~Rôle du portail : l'abonné paie (hypothèse retenue, à confirmer).~~
- [x] ~~Modèle des moyens : compte propre à chaque abonné (hypothèse retenue, à confirmer).~~
- [x] ~~Signature : intégrée au portail, OTP + scellement PAdES.~~
- [ ] Liste définitive des fournisseurs, banques, acquéreurs et APIs disponibles.
- [ ] Règles contractuelles : plafonds, commissions, frais, renouvellement, avenants.
- [ ] Exigences réglementaires locales et responsabilités de chaque partie.
- [x] ~~Fuseau horaire : Africa/Nouakchott.~~
- [x] ~~Frontend : Angular.~~

6. Plan des étapes de réalisation

Le projet se déroule en 10 étapes sur environ 22 semaines, avec la démo du workflow complet dès la semaine 12. Par rapport au découpage de la conception, le moteur des droits passe **avant** l'intégration des fournisseurs : on démontre le workflow avec des paiements simulés pendant que les accès aux APIs arrivent.



feuille de route · 10 étapes, 3 jalons

Durées estimées pour une équipe de 3 développeurs (2 backend, 1 frontend) ; elles sont à ajuster après le cadrage.

Détail des étapes

0. **Cadrage et décisions** (1–2 sem.)
   - Faire confirmer les hypothèses métier retenues (abonné payeur, compte propre, un contrat actif, MRU).
   - Demander les accès sandbox Bankily, Masrvi et passerelle Visa ; choisir l'opérateur SMS.
   - Maquettes des écrans Administrateur et Abonné ; backlog priorisé.
   - Outils : Figma (maquettes), GitLab (dépôt, tickets).
   - Livrable : cahier des charges validé.
1. **Socle technique et authentification** (2 sem.)
   - Créer le projet backend en monolithe modulaire et l'application frontend avec ses 2 espaces.
   - Connexion, rôles ADMIN et ABONNE, MFA admin, journal d'audit de base.
   - Environnements dev et test, pipeline build et tests.
   - Outils : Java 21, Spring Boot 3, Spring Modulith, Angular 20, PrimeNG, Keycloak (OIDC) + Spring Security, PostgreSQL 16, Flyway, Docker Compose, Nginx, GitLab CI.
   - Livrable : accès sécurisé aux deux espaces.
2. **Abonnés et catalogue des moyens** (2 sem.)
   - CRUD abonnés (référence, type, NNI/NIF, coordonnées, statut) et création automatique du compte Keycloak.

- Catalogue avec Bankily, Masrvi et Visa (nom, logo, fournisseur, statut).
- Outils : Spring Data JPA, API d'administration Keycloak, tableaux et formulaires PrimeNG.
- Livrable : l'admin crée un abonné (étape 1 du workflow).

3. **Contrats et PDF** (2 sem.)
   - Modèles de contrat, création d'un contrat, sélection des moyens, dates, plafonds.
   - Génération PDF, stockage, cycle de statuts (brouillon → en attente de signature).
   - Outils : Thymeleaf + OpenHTMLtoPDF, MinIO (S3).
   - Livrable : contrat Bankily + Masrvi + Visa envoyé à l'abonné.
4. **Signature électronique** (2 sem.)
   - Consultation et téléchargement côté abonné, signature par code OTP reçu par SMS.
   - Scellement du PDF, hash, horodatage, conservation des preuves.
   - Outils : EU DSS (scellement PAdES), API SMS de l'opérateur, MinIO.
   - Livrable : contrat signé et archivé.
5. **Moteur des droits et Espace Abonné** (2 sem.)
   - Activation automatique et idempotente : un droit par moyen inscrit.
   - « Mes moyens de paiement » filtré par les droits, contrôle serveur sur chaque appel.
   - Suspension, résiliation et job quotidien d'expiration.
   - Outils : Spring Security (contrôle d'accès), Spring Scheduler + ShedLock, Testcontainers.
   - **Jalon : démo du workflow complet** avec adaptateurs de paiement simulés.
6. **Connecteurs fournisseurs** (3–4 sem.)
   - Interface commune `PaymentProvider` ; adaptateurs Bankily, Masrvi, passerelle Visa (page hébergée, tokenisation).
   - Webhooks signés, anti-rejeu.
   - Outils : Spring WebClient, HashiCorp Vault (clés API), WireMock (tests).
7. **Paiements et transactions** (2 sem.)
   - Initiation avec clé d'idempotence, statut interne vs statut fournisseur, reçus, historique.
   - Rapprochement des paiements en attente.
   - Outils : PostgreSQL (contraintes d'unicité), Spring Scheduler, OpenHTMLtoPDF (reçus).
   - **Jalon : paiements réels en sandbox** sur les 3 moyens.
8. **Notifications, audit et reporting** (2 sem.)
   - SMS et email (contrat disponible, activation, paiement), tableaux de bord Admin et Abonné.

- Outils : Spring Mail, API SMS, graphiques PrimeNG, Prometheus + Grafana, Loki.

9. **Recette, sécurité et mise en production** (2 sem.)
   - Tests de bout en bout du workflow, test d'intrusion, validation du périmètre PCI DSS (SAQ A).
   - Formation des administrateurs, mise en production.
   - Outils : Playwright, OWASP ZAP, Docker Compose + Nginx (HTTPS).
   - **Jalon : mise en production.**